



UNICAMP



Aula 3: Do Baseline ao Agente

Workflow, ferramentas e ReAct com LangGraph

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, 1 de setembro de 2026

Sumário

Da v1 para a v2

Workflow, ReAct e ferramentas

Comparação com o baseline

Objetivos da aula

- ❖ Revisitar as limitações do baseline do Entregável 1
- ❖ Distinguir workflow determinístico, agente ReAct e híbridos
- ❖ Implementar um fluxo com ferramentas e roteamento em LangGraph
- ❖ Instrumentar a comparação entre baseline e v2 na mesma régua
- ❖ Reconhecer os novos modos de falha que a autonomia introduz
- ❖ Preparar a transição para memória e MCP na Aula 4

De onde partimos?

Na Aula 2, o estudo de caso tinha a forma:

Documento + Pergunta $\rightarrow$ LLM $\rightarrow$ Resposta estruturada

Agora perguntamos:

quando uma única chamada deixa de ser suficiente?

Limitações esperadas do baseline

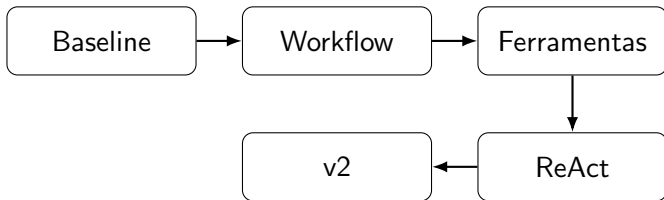
- ❖ Perguntas compostas exigem decomposição
- ❖ Algumas entradas demandam ações diferentes
- ❖ O prompt precisa fazer controle de fluxo demais
- ❖ O sistema não escolhe ferramentas dinamicamente
- ❖ Não há separação clara entre extrair, verificar e sintetizar
- ❖ Falhas intermediárias ficam invisíveis

A pergunta arquitetural da semana

Como transformar o baseline em um sistema agêntico
sem adicionar complexidade desnecessária?

A v2 deve resolver limitações observadas, não apenas acrescentar tecnologias

Trajetória da v2



Nem todo projeto precisa passar por todos esses passos do mesmo modo

Workflow versus agente

Workflow	Agente/ReAct
Fluxo definido pelo desenvolvedor	Próxima ação decidida dinamicamente
Mais previsível	Mais flexível
Mais fácil de testar	Mais difícil de prever comportamentos
Bom para processos conhecidos	Bom para tarefas abertas ou variáveis

Princípio de engenharia (navalha de Ockham)

Use a menor autonomia necessária para resolver o problema

- ❖ Se a ordem das etapas é fixa, um workflow pode bastar
- ❖ Se a ação depende da entrada, roteamento condicional pode bastar
- ❖ Se a ação precisa ser escolhida dinamicamente, ReAct pode ser útil
- ❖ Se há responsabilidades independentes, MAS pode ser justificável depois

Workflow para análise de editais

Uma decomposição simples:

1. classificar a pergunta
2. extrair evidências relevantes
3. verificar se a evidência é suficiente
4. opcionalmente consultar uma ferramenta
5. sintetizar a resposta
6. validar formato e evidência

Ferramentas no estudo de caso

Ferramentas possíveis:

- ❖ consultar prazos
- ❖ consultar critérios de elegibilidade
- ❖ listar documentos obrigatórios
- ❖ buscar anexos
- ❖ calcular cronograma
- ❖ consultar fonte externa ou base institucional

Anatomia de uma ferramenta

```
@tool
def consultar_prazo() -> str:
    """Devolve o trecho do edital que trata de
    prazos de submissao. Use para perguntas sobre datas
    de envio de propostas."""
    return _secao(documento, "PRAZO")
```

- ❖ A **docstring é prompt**: é por ela que o modelo decide se e quando chamar
- ❖ A assinatura tipada define o que o modelo precisa produzir
- ❖ O retorno volta ao contexto: seja curto e específico
- ❖ Falha esperada devolve mensagem tratável, não exceção

A ferramenta precisa fazer trabalho real

Antipadrão

Uma “ferramenta” que devolve sempre a mesma resposta fixa

- ❖ Ela funcionaria igual com o documento errado
- ❖ A comparação com o baseline perde o sentido
- ❖ O sistema parece agêntico, mas não é

No notebook, as ferramentas extraem seções do documento de verdade

ReAct

Observar $\rightarrow$ decidir $\rightarrow$ agir $\rightarrow$ observar $\rightarrow \dots$

- ❖ No ReAct, o modelo não apenas responde: ele decide se precisa usar uma ferramenta e qual ferramenta deve usar
- ❖ Em LangGraph, esse comportamento aparece como um loop controlado entre nó do agente e nó de ferramentas

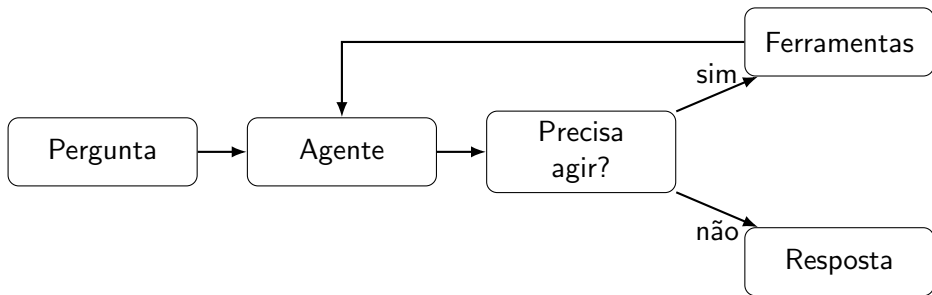
Exemplo de pergunta composta

“Quem pode participar e qual é o prazo para submissão?”

Uma resposta adequada pode exigir:

1. consultar elegibilidade
2. consultar prazo
3. combinar os resultados
4. produzir uma resposta com evidências

Arquitetura da Aula 3



Estado mínimo em LangGraph

```
class State(TypedDict):  
    messages: Annotated[list, add_messages]  
    document: str  
    metrics: dict
```

`add_messages` é um *reducer*: cada nó **acumula** mensagens em vez de sobrescrever o campo

O estado deve conter:

- ❖ mensagens necessárias para o próximo passo
- ❖ o documento ou uma referência ao documento
- ❖ informações instrumentadas para comparação

Todo agente precisa de uma condição de parada

Um loop agente-ferramentas pode não terminar sozinho

- ❖ `recursion_limit` interrompe após N passos
- ❖ Sem freio, o custo é limitado apenas pela cota da API
- ❖ O limite atingido é um **modo de falha**, e deve ser registrado

Autonomia sem limite não é sofisticação; é ausência de projeto

Novos modos de falha da v2

- ❖ Ferramenta certa, argumento errado
- ❖ Ferramenta desnecessária (custo sem ganho)
- ❖ Ferramenta necessária não chamada
- ❖ Erro dentro da ferramenta, engolido silenciosamente
- ❖ Loop até o limite de recursão
- ❖ Resposta final que ignora o que a ferramenta devolveu

Nenhum deles existia em nosso baseline, e a análise de erros similares precisa aparecer na resolução do Entregável 2

Preservar a régua

Para atribuir a diferença à arquitetura, mantenha fixos:

- ❖ o mesmo documento e as mesmas entradas
- ❖ o mesmo conjunto de casos congelado do Entregável 1
- ❖ o **mesmo modelo** e a mesma temperatura
- ❖ o mesmo esquema de saída
- ❖ as mesmas funções de verificação

Trocar o modelo entre v1 e v2 inviabiliza a conclusão: não se sabe o que causou a diferença

O que comparar na v2?

- ❖ A qualidade melhorou nos mesmos casos de teste?
- ❖ A v2 resolveu perguntas compostas que o baseline errava?
- ❖ Quantas chamadas ao LLM foram feitas?
- ❖ Quantas chamadas a ferramentas foram feitas?
- ❖ A latência aumentou? Quanto?
- ❖ Surgiram novos modos de falha?

Medidas para comparar v1 e v2

Medida	Uso
Acerto/erro por caso	Comparar qualidade mínima
Latência	Medir custo temporal da arquitetura
Chamadas ao LLM	Aproximar custo financeiro
Chamadas a ferramentas	Entender comportamento agêntico
Erros de ferramenta	Identificar novos modos de falha

Como ler os números

Com 5 casos automáticos, uma diferença de acerto vale 20 pontos percentuais

O que se pode afirmar:

- ❖ “a v2 acertou o caso composto que a v1 errava”: evidência direta
- ❖ “a latência mediana passou de X para Y”: medida, não estimada

O que **não** se pode afirmar a partir de 5 casos:

- ❖ “a v2 é melhor”
- ❖ Se a v2 empatar ou piorar, isso é resultado, e deve ser reportado como tal

Atividade prática

No notebook da Aula 3:

1. criamos ferramentas para prazos, elegibilidade e documentos
2. conectamos as ferramentas a um agente com LangGraph
3. executamos perguntas simples e compostas
4. inspecionamos tool calls e mensagens intermediárias
5. comparamos a v2 com o baseline

Ponte para a Aula 4

Aula 3: o sistema sabe **agir**

Aula 4: o sistema precisa **manter contexto** e organizar **integrações externas**

Memória + MCP